Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «Избыточное кодирование данных в информационных системах. Код Хемминга. Итеративные коды»

Выполнил:
Студент 4-го курса
Группы АС-66
Осовец А. О.
Проверил:
Хацкевич А. С.

Брест 2026

Цель работы: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга и итеративных кодов.

Ход работы

Вариант 12

1. Код Хемминга (файл «Лабораторная работа №2 (ч.1)»)

Задачи:

- на основе информационного сообщения из текстового файла (русский и английский алфавит, служебные символы и цифры) сформировать сообщение в двоичном виде длиной не менее 16 бит;
- построить проверочную матрицу Хемминга для минимального кодового расстояния $d_{min} = 3$ и 4;
- вычислить избыточные символы X_r, принять слово с 0, 1 и 2 ошибками в случайных позициях;
- для принятого слова $Y_n = Y_k$, Y_r вычислить Y_r' и синдром S; сформировать вектор ошибки E_n, исправить одиночную ошибку и проанализировать двойную;
- вывести в файл и на экран сообщение, k, r, n, $H_{n,k}$, X_n, X_r, Y_n, Y_r, Y_r', S, E_n; программа не должна зависеть от длины сообщения.

2. Итеративные коды (файл «Лабораторная работа №2 (ч.2)»)

Задачи:

- вписывать двоичное слово X_k длиной k в двумерную или трёхмерную матрицу по варианту;
- вычислять проверочные биты по двум, трём, четырём и пяти группам паритетов;
- формировать кодовое слово X_n, генерировать ошибку произвольной кратности i в случайных позициях (слово Y_n);
- определять местоположение ошибок и исправлять их (слово Y_n');
- оценить корректирующую способность отношениями N2/N1 (кратность определена верно) и N3/N1 (все ошибки исправлены).

Постановка задачи для варианта 12

Сообщение для кода Хемминга — строка «КиСБ lab2!» из файла message.txt (10 символов). Символы переводятся в кодировку Windows-1251 по 8 бит, поэтому k = 80. Исследуются оба значения минимального кодового расстояния: $d_{min} = 3$ и $d_{min} = 4$.

Таблица 5.2 содержит пять вариантов, поэтому номеру 12 по списку соответствует вариант ((12 − 1) mod 5) + 1 = 2: длина информационного слова k = 20 бит. Слово X_k по умолчанию — первые 20 бит того же сообщения: 11001010111010001101.

Таблица 1 — Параметры итеративных кодов варианта 2 (k = 20)

Раскладка	k_1	k_2	z	Группы паритетов
2D	4	5	–	2; 3
2D	2	10	–	2; 3
3D	2	5	2	2; 3; 4; 5
3D	2	2	5	2; 3; 4; 5

Теоретические положения

Линейный блочный код (n, k) добавляет к k информационным битам r = n − k проверочных. Код с минимальным расстоянием d обнаруживает $t_о$ ошибок и исправляет $t_и$ ошибок: для d = 3 $t_о = t_и = 1$, для d = 4 $t_о = 2$, $t_и = 1$ (разд. 4.1.1 методички).

Для кода Хемминга с $d_{min} = 3$ число проверочных битов — минимальное r, при котором $2^r \geq k + r + 1$. Проверочная матрица H = [P' | I]: столбцы подматрицы P' — r-битные числа веса 2, затем 3 и т. д. в порядке возрастания (их C_r^{wt} для каждого веса), I — единичная матрица r × r. Избыточные символы равны $X_r = P' \cdot X_k$. Синдром $S = Y_r + Y_r' = H \cdot Y_n^T$ равен сумме столбцов H с номерами ошибочных битов. Если S совпадает со столбцом j, ошибка одиночная в бите j. Тогда $E_n = (0 \ldots 1 \ldots 0)$ и $X_n = Y_n + E_n$ (все сложения — по модулю 2).

Для $d_{min} = 4$ к H добавляется строка из единиц (4.8). Чтобы матрица осталась канонической, эта строка заменяется суммой всех строк H'. Все столбцы канонической H' имеют нечётный вес. Поэтому синдром нечётного веса указывает на одиночную ошибку, а ненулевой синдром чётного веса — на двойную: она обнаруживается, но не исправляется.

Итеративный (прямоугольный, композиционный) код записывает X_k в таблицу $k_1 \times k_2$ или в z таких плоскостей. Проверочные биты — паритеты строк (H), столбцов (V), диагоналей (D), антидиагоналей (A) и паритеты вдоль оси Z. Код из двух групп с символом X_{hv} имеет $d_{min} = 4$, без X_{hv} — 3. Простейшая необнаруживаемая ошибка — четыре бита в вершинах прямоугольника (рис. 5.2, г). Дополнительные группы паритетов разрешают такие конфигурации ценой роста избыточности.

Особенности реализации

Программа написана на C++17 одним файлом main.cpp и не использует динамических контейнеров и выделения памяти в куче: все буферы имеют

фиксированный размер. Сообщение может содержать до 256 символов ($k \leq 2048$ бит, $r \leq 13$), поэтому программа не привязана к длине сообщения; это проверено на 2- и 256-символьных файлах. Текст читается из файла в UTF-8 и преобразуется в Windows-1251. Весь вывод дублируется в файл lab2_output.txt.

Столбец H хранится как 16-битное число. X_r вычисляется как XOR столбцов P' по единичным битам X_k через битовую маску, без ветвлений. Поиск столбца, равного синдрому, и инверсия ошибочного бита выполняются полным проходом по всем n столбцам без переходов, зависящих от данных. Построение H проверяется на примере H(7,4) из методички.

Для итеративного кода каждая проверка задаётся 64-битной маской информационных битов, и для каждого бита хранится маска проверок, в которые он входит. В 2D диагонали обычные ($k_1 + k_2 - 1$ проверок, рис. 5.3), в 3D — объединённые циклические, $\max(k_1, k_2)$ на плоскость (рис. 5.4). Число групп g — это первые g групп списка H, V, D, A, Z. Кодирование проверяется на примере рис. 5.1: $X_k = 011101111 \rightarrow 0010011$.

Декодер итеративного кода выполняет итеративное инвертирование битов. На каждом шаге для каждого информационного бита считается число ошибочных проверок, в которые он входит. Бит с наибольшим числом инвертируется, если ошибочны больше половины его групп. Проверки, оставшиеся ошибочными, относятся к ошибкам в самих проверочных битах. Число исправленных позиций — оценка кратности ошибки. Если её чётность противоречит X_{hv}, слово помечается как «обнаружено, но не исправлено». Позиции ошибок задаёт генератор SplitMix64 с фиксированным seed, поэтому результаты воспроизводимы.

Листинги ключевых фрагментов (main.cpp)

Листинг 1 — Построение матрицы Хемминга, кодирование и декодирование

```
// H = [P' | I]: columns of P' are r-bit numbers of weight 2, 3, ... taken in
// ascending order (listing 4.2). For d_min = 4 the row of ones from (4.8) is
// replaced by the sum of all rows, which keeps H in canonical form.
void build_hamming(HammingCode& code, unsigned int k, unsigned int d_min) {
    const unsigned int r = hamming_check_bits(k);
    unsigned int filled = 0U;
    for (unsigned int weight = 2U; weight <= r && filled < k; ++weight) {
        for (unsigned int value = 3U; value < (1U << r) && filled < k; ++value) {
            if (popcount64(value) == weight) {
                code.column[filled++] = static_cast<uint16_t>(value);
            }
        }
    }
    for (unsigned int row = 0U; row < r; ++row) {
        code.column[k + row] = static_cast<uint16_t>(1U << (r - 1U - row));
    }
    code.k = k;
    code.r = r;
    code.n = k + r;
    code.d_min = 3U;
    if (d_min == 4U) {
        for (unsigned int j = 0U; j < code.n; ++j) {
            const unsigned int value = code.column[j];
```

```
            code.column[j] = static_cast<uint16_t>((value << 1U) | (parity64(value) ^ 1U));
        }
        code.column[code.n] = 1U;
        code.r = r + 1U;
        code.n = k + r + 1U;
        code.d_min = 4U;
    }
}

uint16_t hamming_checks(const HammingCode& code, const uint8_t* info) {
    uint16_t checks = 0U;
    for (unsigned int j = 0U; j < code.k; ++j) {
        checks ^= static_cast<uint16_t>(code.column[j] & (0U - static_cast<unsigned int>(info[j])));
    }
    return checks;
}

// Corrects the word in place; the column search and the bit flip have no
// data-dependent branches.
HammingDecoding hamming_decode(const HammingCode& code, uint8_t* word) {
    HammingDecoding result{ 0U, 0U, 0U, 0U, DecodeStatus::kNoError };
    for (unsigned int row = 0U; row < code.r; ++row) {
        result.received_checks = static_cast<uint16_t>((result.received_checks << 1U) | word[code.k + row]);
    }
    result.recomputed_checks = hamming_checks(code, word);
    result.syndrome = static_cast<uint16_t>(result.received_checks ^ result.recomputed_checks);

    unsigned int found = 0U;
    unsigned int position = 0U;
    for (unsigned int j = 0U; j < code.n; ++j) {
        const unsigned int match = static_cast<unsigned int>(code.column[j] == result.syndrome);
        position |= j & (0U - match);
        found |= match;
    }
    for (unsigned int j = 0U; j < code.n; ++j) {
        word[j] ^= static_cast<uint8_t>(found & static_cast<unsigned int>(j == position));
    }
    result.position = position;
    if (result.syndrome == 0U) {
        result.status = DecodeStatus::kNoError;
    }
    else if (found != 0U) {
        result.status = DecodeStatus::kCorrected;
    }
    else {
        result.status = DecodeStatus::kDetected;
    }
    return result;
}
```

Листинг 2 — Построение итеративного кода и декодер

```
// 2D codes use plain diagonals (k1 + k2 - 1 checks, fig. 5.3); 3D codes use
// combined (cyclic) diagonals, max(k1, k2) checks per plane (fig. 5.4).
unsigned int line_index(const IterativeCode& code, unsigned int group, unsigned int plane,
    unsigned int row, unsigned int col) {
    const bool cyclic = code.z > 1U;
    switch (group) {
    case kRows:
        return plane * code.k1 + row;
    case kColumns:
        return plane * code.k2 + col;
    case kDiagonals:
        return plane * code.span +
            (cyclic ? (col + code.span - row) % code.span : col + code.k1 - 1U - row);
    case kAntiDiagonals:
        return plane * code.span + (cyclic ? (row + col) % code.span : row + col);
    default:
        return row * code.k2 + col;
    }
}

bool build_iterative(IterativeCode& code, const VariantRow& variant, unsigned int groups, bool with_total) {
    code.k1 = variant.k1;
    code.k2 = variant.k2;
    code.z = variant.z;
    code.groups = groups;
    code.with_total = with_total;
    code.k = code.k1 * code.k2 * code.z;
    code.span = code.z > 1U ? (code.k1 > code.k2 ? code.k1 : code.k2) : code.k1 + code.k2 - 1U;
    if (code.k > kMaxIterativeBits || groups < 2U || groups > kGroupCount ||
        (code.z == 1U && groups > 3U)) {
        return false;
    }
    const unsigned int lines[kGroupCount] = {
```

```
        code.z * code.k1, code.z * code.k2, code.z * code.span, code.z * code.span, code.k1 * code.k2
    };
    code.checks = 0U;
    for (unsigned int group = 0U; group < groups; ++group) {
        code.group_begin[group] = code.checks;
        code.checks += lines[group];
    }
    for (unsigned int group = groups; group <= kGroupCount; ++group) {
        code.group_begin[group] = code.checks;
    }
    if (code.checks > kMaxIterativeChecks) {
        return false;
    }
    code.r = code.checks + static_cast<unsigned int>(with_total);
    code.n = code.k + code.r;
    memset(code.check_mask, 0, sizeof(code.check_mask));
    memset(code.cover, 0, sizeof(code.cover));
    for (unsigned int plane = 0U; plane < code.z; ++plane) {
        for (unsigned int row = 0U; row < code.k1; ++row) {
            for (unsigned int col = 0U; col < code.k2; ++col) {
                const unsigned int bit = info_index(code, plane, row, col);
                for (unsigned int group = 0U; group < groups; ++group) {
                    const unsigned int check = code.group_begin[group] + line_index(code, group, plane, row, col);
                    code.check_mask[check] |= 1ULL << bit;
                    code.cover[bit] |= 1ULL << check;
                }
            }
        }
    }
    return true;
}

// Iterative bit flipping: an information bit is flipped while more than half
// of its parity groups fail; failed checks that remain point to errors in
// the check bits themselves. X_hv confirms the parity of the estimated
// error multiplicity.
IterativeDecoding iterative_decode(const IterativeCode& code, const IterativeWord& received) {
    IterativeDecoding result{ received, 0ULL, 0U, 0ULL, 0ULL, 0U, 0U, false };
    uint64_t syndrome = received.checks ^ iterative_checks(code, received.info);
    result.syndrome = syndrome;
    if (code.with_total) {
        result.total_syndrome = parity64(received.info) ^ parity64(received.checks) ^ received.total;
    }
    for (unsigned int step = 0U; step < code.k; ++step) {
        unsigned int best_votes = 0U;
        unsigned int best_bit = 0U;
        for (unsigned int bit = 0U; bit < code.k; ++bit) {
            const unsigned int votes = popcount64(syndrome & code.cover[bit]);
            if (votes > best_votes) {
                best_votes = votes;
                best_bit = bit;
            }
        }
        if (2U * best_votes <= code.groups) {
            break;
        }
        result.word.info ^= 1ULL << best_bit;
        result.flipped_info ^= 1ULL << best_bit;
        syndrome ^= code.cover[best_bit];
    }
    result.flipped_checks = syndrome;
    result.word.checks ^= syndrome;
    result.corrections = popcount64(result.flipped_info) + popcount64(result.flipped_checks);
    if (code.with_total && result.total_syndrome != (result.corrections & 1U)) {
        if (result.corrections == 0U) {
            result.word.total ^= 1U;
            result.flipped_total = 1U;
            result.corrections = 1U;
        }
        else {
            result.rejected = true;
        }
    }
    return result;
}
```

Результаты работы программы

Сборка и запуск: g++ -std=c++17 -O2 main.cpp -o kisb_lab2_variant12; аргументы командной строки — файл сообщения (по умолчанию message.txt), seed генератора ошибок (12) и слово X_k для ч.2. Полный вывод приложен в sample_output.txt.

```
$ ./kisb_lab2_variant12 message.txt 12
Laboratory work 2, variant 12 (table 5.2 variant 2, k = 20)
Message source: message.txt; seed = 12; report file: lab2_output.txt
Message: "КиСБ lab2!" (10 symbols, Windows-1251, 80 bits)
Self-checks: textbook H(7,4) PASS; fig. 5.1 iterative example PASS

================ Part 1. Hamming code, d_min = 3 ================
k = 80, r = 7, n = 87, code rate k/n = 0.9195
2^7 = 128 >= k + r + 1 = 88; t_o = 1, t_i = 1 (section 4.1.1)
P' columns: C(7,2) = 21 -> 21 used; C(7,3) = 35 -> 35 used; C(7,4) = 35 -> 24 used;
  H =    00000000000000011111100000000000000000001111111111111110000000000000000111111111|1000000
         00000000001111100000100000000011111111110000000000111110000011111111110000000000|0100000
         00000011110000100001000001111110000001111000000111100001011110000111111000011111|0010000
         00011100010001000010001110001110001110001000111000100010101110111000111011100011|0001000
         01100100100010000100010110110010110010010011001001000100110111011011001101101100|0000100
         10101001000100001000011011010101010100100101010010001000111011101101010110110101|0000010
         11010010001000010000011101101001101001000110100100010000111011101101001110110010|0000001
  X_k =  11001010111010001101000111000001001000001101100011000010110001000110010001000001
  X_r =  0101101
  X_n =  11001010111010001101000111000001001000001101100011000010110001000110010001000001|0101101

-- 0 error(s), random positions: none
  Y_n =  11001010111010001101000111000001001000001101100011000010110001000110010001000001|0101101
  Y_k =  11001010111010001101000111000001001000001101100011000010110001000110010001000001
  Y_r =  0101101
  Y_r'=  0101101
  S   =  0000000
  E_n =  00000000000000000000000000000000000000000000000000000000000000000000000000000000|0000000
  Y+E =  11001010111010001101000111000001001000001101100011000010110001000110010001000001|0101101
analysis: S = 0, Y_n is a code word; no error detected.
Y_n + E_n == X_n
decoded message: "КиСБ lab2!"

-- 1 error(s), random positions: 47
  Y_n =  11001010111010001101000111000001001000001101110011000010110001000110010001000001|0101101
                                                          ^
  Y_k =  11001010111010001101000111000001001000001101110011000010110001000110010001000001
  Y_r =  0101101
  Y_r'=  1100001
  S   =  1001100
  E_n =  00000000000000000000000000000000000000000000100000000000000000000000000000000000|0000000
  Y+E =  11001010111010001101000111000001001000001101100011000010110001000110010001000001|0101101
analysis: S equals column 47 of H -> single error in bit 47, corrected.
Y_n + E_n == X_n
decoded message: "КиСБ lab2!"
```

Рисунок 1 — Код Хемминга, $d_{min} = 3$: матрица H, кодовое слово, приём без ошибки и с одиночной ошибкой

```
d_min = 3:
-- 2 error(s), random positions: 49, 68
  Y_n =  1100101011101000110100011100000100100000011011001110000101100010001000100010000 1|0101101
                                                                  ^                  ^
  Y_k =  1100101011101000110100011100000100100000011011001110000101100010001000100010000 1
  Y_r =  0101101
  Y_r'=  1001001
  S   =  1100100
  E_n =  0000000000000000000000000000000000000000000000000000000000000000000000000000000 0|0000000
  Y+E =  1100101011101000110100011100000100100000011011001110010101100010001000100010000 1|0101101
analysis: S equals column 54 of H -> single error in bit 54, corrected.
Y_n + E_n != X_n  (miscorrection: 2 errors exceed t_i = 1)
decoded message: "КиСБ leb"!"

d_min = 4:
-- 2 error(s), random positions: 7, 75
  Y_n =  1100100011101000110100011100000100100000011011000110000101100010001100100000000 1|01011011
               ^                                                                          ^
  Y_k =  1100100011101000110100011100000100100000011011000110000101100010001100100000000 1
  Y_r =  01011011
  Y_r'=  11100101
  S   =  10111110
  E_n =  not formed: the syndrome points to no single bit
analysis: S != 0 and weight(S) is even -> double error detected, cannot be corrected.
received message: "ИиСБ lab2?"
```

Рисунок 2 — Две ошибки: ложное исправление при $d_{min} = 3$ и обнаружение при $d_{min} = 4$

Для k = 80 получено r = 7 ($2^7 = 128 \geq 88$): код (87, 80) со скоростью 0,920. Столбцы P' заняли все $C_7^2 = 21$ числа веса 2, все $C_7^3 = 35$ веса 3 и 24 из 35 чисел веса 4. Для $d_{min} = 4$ код (88, 80), скорость 0,909. Одиночная ошибка в бите 47 (рис. 1) даёт синдром 1001100, равный 47-му столбцу H. Вектор E_n с единицей в этой позиции восстанавливает X_n и текст «КиСБ lab2!». При двух ошибках код с $d_{min} = 3$ нашёл столбец 54, равный сумме столбцов 49 и 68, и испортил третий бит: сообщение стало «КиСБ leb"!». Код с $d_{min} = 4$ получил синдром чётного веса и правильно отказался исправлять.

Таблица 2 — Статистика декодирования кода Хемминга (10 000 случайных слов на строку)

d_{min}	Кратность	Обнаружено	Исправлено верно	Ложное исправление	Обнаружено, не исправлено
3	0	0	10000	0	0
3	1	10000	10000	0	0
3	2	10000	0	7200	2800
3	3	9926	0	6819	3107
4	0	0	10000	0	0
4	1	10000	10000	0	0
4	2	10000	0	0	10000
4	3	10000	0	6873	3127

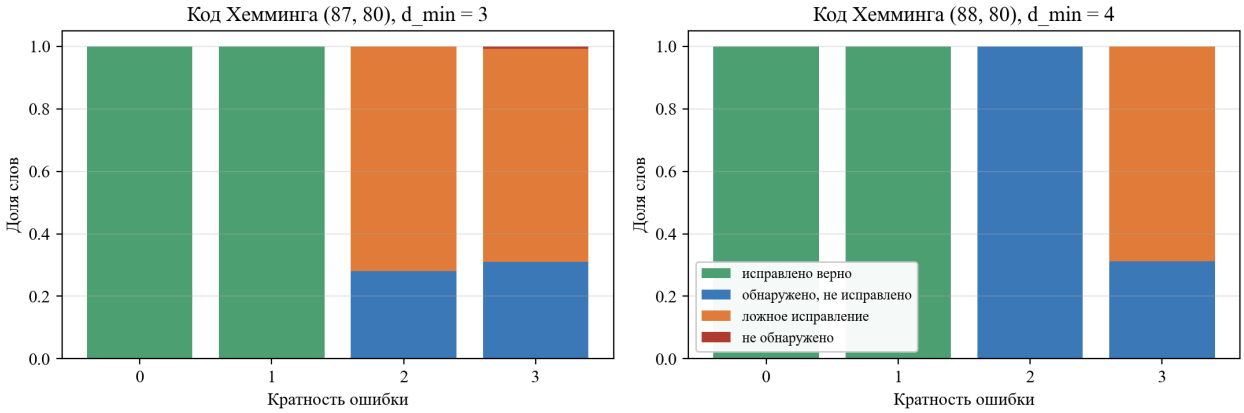


Рисунок 3 — Исходы декодирования кода Хемминга в зависимости от кратности ошибки

```
=== 4x5 (k1 = 4, k2 = 5, z = 1), groups: H V D + X_hv ===
k = 20, r = 18, n = 38, rate k/n = 0.5263
    1 1 0 0 1 | 1
    0 1 0 1 1 | 1
    1 0 1 0 0 | 0
    0 1 1 0 1 | 1
    ----------+
    0 1 0 1 1   <- V
    D: 0 0 1 1 0 1 1 1
  X_hv = 0
  X_k =  11001010111010001101
  X_r =  1101|01011|00110111|0
  X_n =  11001010111010001101|1101|01011|00110111|0
-- 1 error(s) at 23
  Y_n =  11001010111010001101|1111|01011|00110111|0
                                ^
  S   =  0010|00000|00000000|1
  Y_n'=  11001010111010001101|1101|01011|00110111|0
                                ^
located: 23; X_n == Y_n'
-- 2 error(s) at 9, 36
  Y_n =  11001010011010001101|1101|01011|00110101|0
                ^                               ^
  S   =  0100|00010|00000110|0
  Y_n'=  11001010111010001101|1101|01011|00110111|0
                ^                               ^
located: 9, 36; X_n == Y_n'
-- 3 error(s) at 12, 33, 35
  Y_n =  11001010111110001101|1101|01011|00100011|0
                   ^                           ^ ^
  S   =  0010|01000|00110100|1
  Y_n'=  11001010111010001101|1101|01011|00110111|0
                   ^                           ^ ^
located: 12, 33, 35; X_n == Y_n'
```

Рисунок 4 — Итеративный код 4×5 с группами H, V, D и X_{hv}: таблица паритетов и исправление 1–3 ошибок

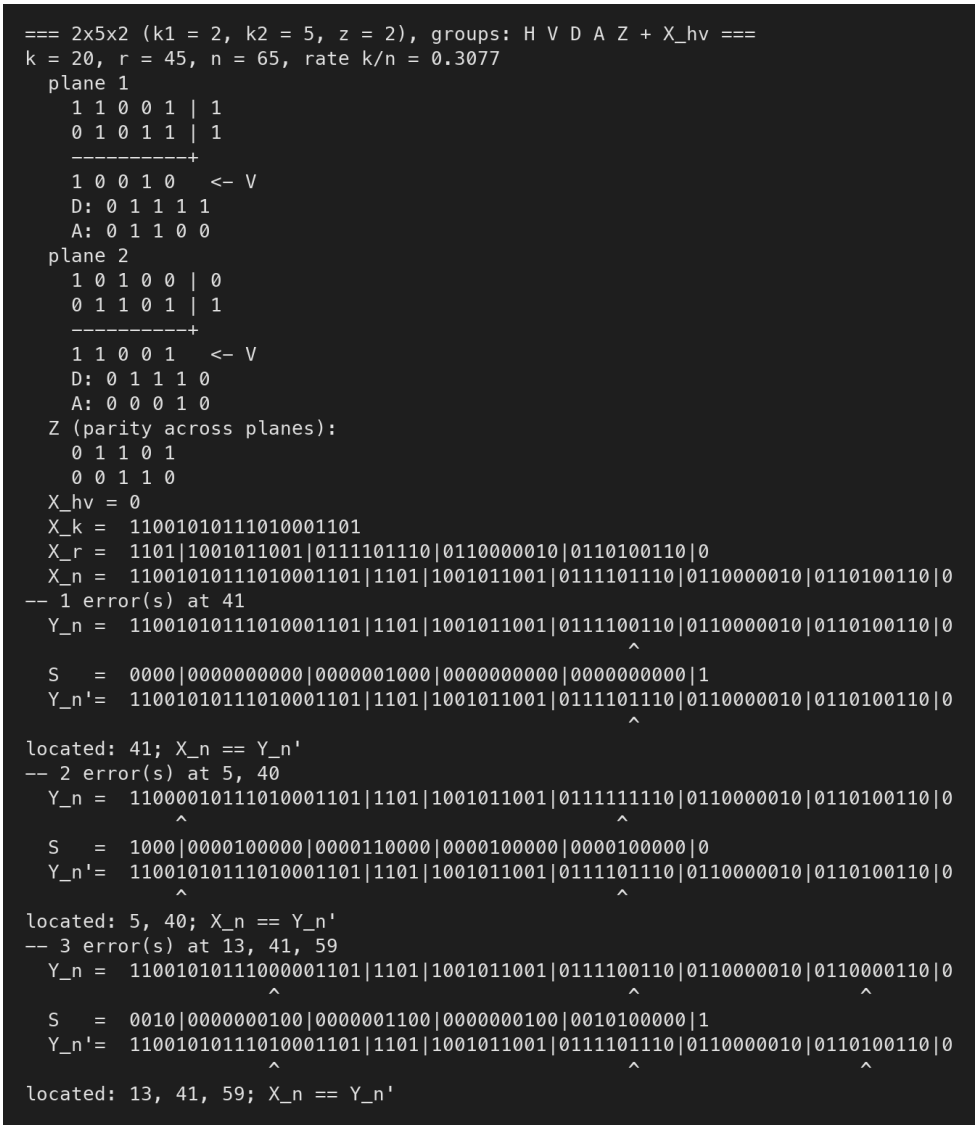


Рисунок 5 — Трёхмерный итеративный код 2×5×2 с пятью группами паритетов

Таблица 3 — Корректирующая способность итеративных кодов (N1 = 10 000 слов на ячейку)

Схема	g	X_{hv}	r	n	k/n	i = 1 N2/N 1	i = 1 N3/N 1	i = 2 N2/N 1	i = 2 N3/N 1	i = 3 N2/N 1	i = 3 N3/N 1	i = 4 N2/N 1	i = 4 N3/N 1
4×5	2	нет	9	29	0,690	1,000	1,000	0,852	0,361	0,516	0,059	0,185	0,003
4×5	2	да	10	30	0,667	1,000	1,000	0,799	0,336	0,467	0,052	0,155	0,004
4×5	3	нет	17	37	0,541	1,000	1,000	0,939	0,851	0,850	0,555	0,782	0,247
4×5	3	да	18	38	0,526	1,000	1,000	0,896	0,813	0,790	0,521	0,703	0,229
2×10	2	нет	12	32	0,625	1,000	1,000	0,881	0,389	0,650	0,096	0,471	0,024
2×10	2	да	13	33	0,606	1,000	1,000	0,827	0,372	0,585	0,085	0,415	0,020
2×10	3	нет	23	43	0,465	1,000	1,000	0,979	0,911	0,924	0,719	0,854	0,487
2×10	3	да	24	44	0,455	1,000	1,000	0,935	0,870	0,858	0,674	0,776	0,444
2×5×2	2	нет	14	34	0,588	1,000	1,000	0,895	0,681	0,694	0,279	0,455	0,076
2×5×2	2	да	15	35	0,571	1,000	1,000	0,841	0,637	0,632	0,254	0,402	0,065
2×5×2	3	нет	24	44	0,455	1,000	1,000	0,978	0,911	0,943	0,752	0,889	0,536
2×5×2	3	да	25	45	0,444	1,000	1,000	0,935	0,874	0,877	0,693	0,812	0,489

Схема	g	X_{hv}	r	n	k/n	i = 1 N2/N 1	i = 1 N3/N 1	i = 2 N2/N 1	i = 2 N3/N 1	i = 3 N2/N 1	i = 3 N3/N 1	i = 4 N2/N 1	i = 4 N3/N 1
2×5×2	4	нет	34	54	0,370	1,000	1,000	1,000	1,000	0,982	0,982	0,940	0,935
2×5×2	4	да	35	55	0,364	1,000	1,000	0,964	0,964	0,930	0,930	0,872	0,868
2×5×2	5	нет	44	64	0,312	1,000	1,000	1,000	1,000	1,000	0,996	0,999	0,982
2×5×2	5	да	45	65	0,308	1,000	1,000	0,967	0,967	0,952	0,948	0,935	0,920
2×2×5	2	нет	20	40	0,500	1,000	1,000	0,920	0,859	0,780	0,637	0,591	0,379
2×2×5	2	да	21	41	0,488	1,000	1,000	0,877	0,823	0,718	0,588	0,533	0,346
2×2×5	3	нет	30	50	0,400	1,000	1,000	1,000	0,939	0,995	0,817	0,978	0,669
2×2×5	3	да	31	51	0,392	1,000	1,000	0,961	0,903	0,935	0,772	0,906	0,621
2×2×5	4	нет	40	60	0,333	1,000	1,000	0,994	0,989	0,980	0,960	0,953	0,912
2×2×5	4	да	41	61	0,328	1,000	1,000	0,964	0,958	0,931	0,911	0,886	0,847
2×2×5	5	нет	44	64	0,312	1,000	1,000	1,000	1,000	0,999	0,995	0,994	0,972
2×2×5	5	да	45	65	0,308	1,000	1,000	0,969	0,969	0,948	0,943	0,932	0,916

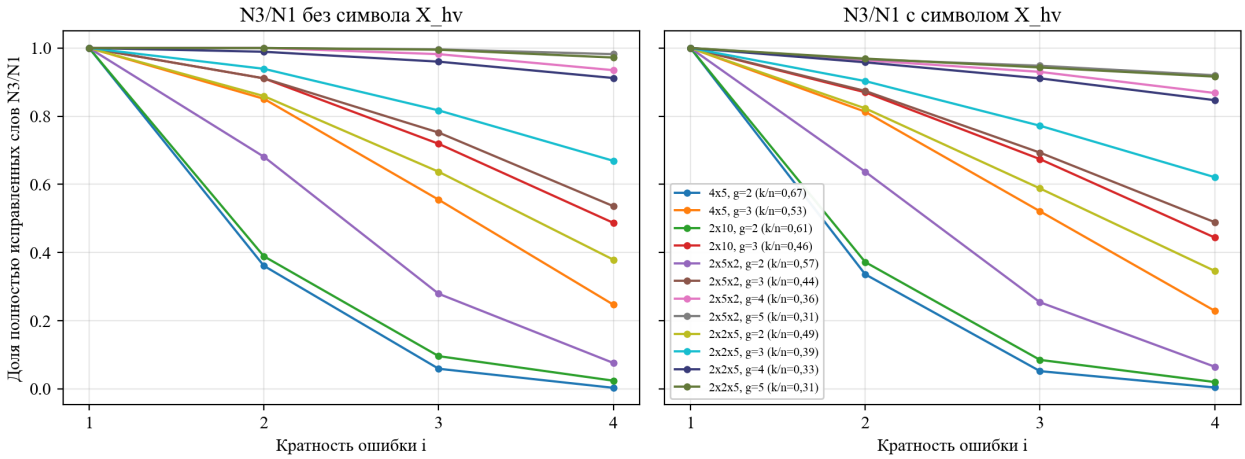


Рисунок 6 — Доля полностью исправленных слов N3/N1 в зависимости от кратности ошибки

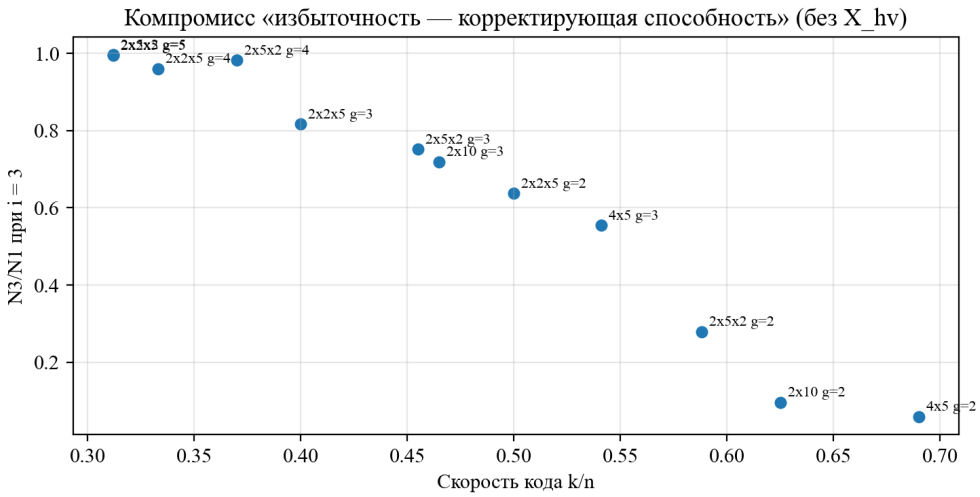


Рисунок 7 — Скорость кода и доля исправленных тройных ошибок

Все конфигурации исправляют 100% одиночных ошибок. С двумя группами (H, V) двойная ошибка исправляется лишь в 36,1% случаев для 4×5. Причина: две ошибки в одной строке или столбце не нарушают паритет этой строки (рис. 5.2, б, в), и декодер принимает их за ошибки проверочных битов. Диагональная группа D разрешает такие случаи: для 4×5 доля исправленных двойных ошибок растёт до 85,1%. Трёхмерный код 2×5×2 с пятью группами исправляет 99,6% тройных и 98,2% четверных ошибок, но его скорость 0,312.

При одинаковом числе групп квадратнее раскладка — меньше избыточность: 4×5 требует r = 9, а 2×10 — r = 12. Для плоскостей 2×2 объединённые диагонали и антидиагонали совпадают, так как $j - i \equiv i + j \pmod 2$. Поэтому четвёртая группа у 2×2×5 дублирует третью: она добавляет 10 бит, а N3/N1 при i = 3 растёт с 0,817 до 0,960 только за счёт двойного веса диагональной проверки в правиле большинства.

Символ X_{hv} не увеличивает долю исправленных слов, а превращает часть ложных исправлений в обнаруженные ошибки. Для 4×5 с двумя группами при i = 2 ложно исправляется 63,9% слов без X_{hv} и 46,3% с X_{hv}: ещё 20,1% помечаются как неисправимые. Плата за это — ошибка в самом бите X_{hv} вместе с другой ошибкой: такое слово тоже отклоняется, поэтому у сильных кодов N3/N1 с X_{hv} на 3–4% ниже.

Ответы на контрольные вопросы

Код Хемминга.

1. Избыточное кодирование добавляет к данным r проверочных битов, зависящих от информационных. Тогда искажённое слово перестаёт быть кодовым: ошибку можно обнаружить по синдрому, а при достаточном кодовом расстоянии — исправить.

2. r — минимальное число, при котором $2^r \geq k + r + 1$, то есть r растёт примерно как $\log_2 k$. Относительная избыточность r/n с ростом k падает. Время передачи по сети возрастает в n/k раз: для k = 4 — в 1,75 раза, для k = 80 — лишь в 1,09 раза. Платой за длинное слово остаётся исправление только одной ошибки на всё слово.

Таблица 4 — Число проверочных битов кода Хемминга

k	r (d=3)	n (d=3)	r (d=4)	n (d=4)	r/n (d=3)
4	3	7	4	8	0,429
6	4	10	5	11	0,400
8	4	12	5	13	0,333
9	4	13	5	14	0,308
10	4	14	5	15	0,286

k	r (d=3)	n (d=3)	r (d=4)	n (d=4)	r/n (d=3)
15	5	20	6	21	0,250
16	5	21	6	22	0,238
80	7	87	8	88	0,080

3. Матрицы строятся программой для любого k тем же алгоритмом (листинг 1). Например, для k = 4:

```
d = 3:  H(7,4) =
0111 | 100
1011 | 010
1101 | 001

d = 4:  H(8,4) =
0111 | 1000
1011 | 0100
1101 | 0010
1110 | 0001
```

Для остальных k из вопроса (6, 8, 9, 10, 15, 16) P' составляется так же: сначала все столбцы веса 2, затем веса 3 и т. д., значения r приведены в таблице 4.

4. Код простой чётности (k + 1, k) имеет один проверочный бит, его H = [1 1 … 1 | 1]. Любые два различных кодовых слова отличаются минимум в двух позициях: изменение одного информационного бита меняет и бит чётности. Поэтому $d_{min} = 2$: код обнаруживает любую нечётную ошибку, но не исправляет ни одной.

5. При выборе предпочтительны столбцы меньшего веса. В H меньше единиц, поэтому каждый проверочный бит считается по меньшему числу информационных: кодер и декодер проще, требуют меньше операций XOR и работают быстрее. По этой же причине алгоритм начинает с веса 2.

Итеративные коды.

1. Параметры итеративного кода: размерность таблицы $k_1 \times k_2$ ($\times$ z), $k = k_1 k_2 z$, число групп паритетов и r — сумма проверок по группам ($k_1 + k_2$ для двух групп, +1 с X_{hv}). Также это скорость k/n и d_{min}: 3 без X_{hv} и 4 с X_{hv} для двух групп.

2. Код Хемминга оптимален по избыточности (k = 80: r = 7, k/n = 0,92), но исправляет одну ошибку. Итеративный код при k = 20 требует r = 9…45, но прост в реализации, начинает декодирование до приёма всего блока и с 4–5 группами исправляет большинство двойных–четверных ошибок.

Проверочная матрица для 9-битного сообщения по схеме рис. 5.1: без X_{hv} H(15,9); с X_{hv} добавляется строка из единиц (H(16,9)).

```
111000000 | 1000000
000111000 | 0100000
000000111 | 0010000
100100100 | 0001000
010010010 | 0000100
001001001 | 0000010
111111111 | 1111111
```

Проверочная матрица для 9-битного сообщения по схеме рис. 5.3: без X_{hv} H(20,9); с X_{hv} добавляется строка из единиц (H(21,9)).

```
111000000 | 100000000000
000111000 | 010000000000
000000111 | 001000000000
100100100 | 000100000000
010010010 | 000010000000
001001001 | 000001000000
000000100 | 000000100000
000100010 | 000000010000
100010001 | 000000001000
010001000 | 000000000100
001000000 | 000000000010
111111111 | 111111111111
```

Проверочная матрица для 9-битного сообщения по схеме рис. 5.4: без X_{hv} H(21,9); с X_{hv} добавляется строка из единиц (H(22,9)).

```
111000000 | 1000000000000
000111000 | 0100000000000
000000111 | 0010000000000
100100100 | 0001000000000
010010010 | 0000100000000
001001001 | 0000010000000
100010001 | 0000001000000
010001100 | 0000000100000
001100010 | 0000000010000
100001010 | 0000000001000
010100001 | 0000000000100
001010100 | 0000000000010
111111111 | 1111111111111
```

Строки над последней — матрица без X_{hv} (последний столбец тогда отсутствует). Для схемы рис. 5.4 одна плоскость 3 × 3 использует объединённые диагонали и антидиагонали; при нескольких плоскостях добавляется группа Z.

6. Код из двух групп с X_{hv} гарантированно обнаруживает до трёх ошибок (d_{min} = 4). Кроме того, обнаруживается любое нечётное число ошибок в одной строке или столбце. Не обнаруживаются только конфигурации, где каждая строка и каждый столбец содержат чётное число ошибок, например четыре бита в вершинах прямоугольника.

7. При фиксированном k наименьшую относительную избыточность даёт квадрат (в 3D — куб): сумма $k_1 + k_2$ при $k_1 k_2 = k$ минимальна при $k_1 = k_2 = \sqrt{k}$. Например, при k = 16 таблица 4 × 4 требует r = 8, а 2 × 8 — r = 10. Для варианта (k = 20) 4 × 5 даёт r = 9 против 12 у 2 × 10.

Вывод: в ходе работы освоено помехоустойчивое кодирование кодом Хемминга и итеративными кодами. Реализовано построение проверочной матрицы Хемминга для произвольной длины сообщения с d_{min} = 3 и 4. Для сообщения «КиСБ lab2!» (k = 80) получены коды (87, 80) и (88, 80). Одиночные ошибки исправляются в 100% случаев. Двойные при d_{min} = 3 обнаруживаются всегда, но в 72% случаев ложно исправляются; при d_{min} = 4 они всегда обнаруживаются и не искажаются. Для варианта 2 (k = 20)

реализованы двумерные и трёхмерные итеративные коды с 2–5 группами паритетов и итеративный декодер. Корректирующая способность растёт с числом групп ценой избыточности: от k/n = 0,69 (4×5, две группы, исправляются только одиночные ошибки) до k/n = 0,31 (2×5×2, пять групп, исправляется 98,2% четырёхкратных ошибок).